KONGU ENGINEERING COLLEGE

SMART INVENTORY MANAGEMENT SYSTEM FOR CONSTRUCTION SITES

MECHATRONICS AND IOT

EXERCISE – 03

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

SMART INVENTORY MANAGEMENT SYSTEM FOR CONSTRUCTION SITES

1. Project Overview

Introduction

Construction sites require the management of large volumes and varieties of materials — cement, steel bars, bricks, sand, tools, machinery and safety equipment — that move continuously between the central store, various work zones, and sometimes between multiple sites. Manual tracking using paper registers or spreadsheets is slow, error-prone, and provides no real-time visibility.

The Smart Inventory Management System is an IoT-based solution that uses RFID technology to automatically identify materials and equipment as they move in and out of the site store. Each material, tool, or piece of equipment is fitted with a passive RFID tag holding a unique identification number. An RFID reader module positioned at the store gate or checkpoint scans the tag whenever an item passes by.

The ESP8266 NodeMCU, acting as the main controller, reads the tag data from the RFID module, checks it against the registered inventory list, updates the stock count, displays the material name and transaction status on an LCD screen, and uploads the record to a cloud platform over Wi-Fi so that inventory levels can be monitored remotely in real time by site engineers and project managers.

Main Features

- Automatic identification of materials and equipment using RFID tags.
- Real-time inward and outward inventory tracking.
- Local status display on LCD.
- Cloud-based data logging for remote monitoring.
- Reduced manual entry and paperwork.
- Minimized material theft, loss, and misplacement.
- Flags unregistered or unauthorized tag movement.
- IoT-ready system using ESP8266 Wi-Fi connectivity.

2. Problem Statement

Construction sites presently depend largely on manual registers, verbal confirmation, or spreadsheet entries maintained by site supervisors and storekeepers to track incoming and outgoing materials. This approach leads to:

- Delays in verifying available stock.
- Errors and inconsistencies in manual data entry.
- Material theft, pilferage, and misplacement due to the absence of real-time tracking.
- No remote visibility for project managers or contractors who are off site.

- Time-consuming physical stock reconciliation.
- Difficulty tracking tools and equipment movement between work zones.

Therefore, an automated, real-time system is required to track material movement accurately and make inventory data available remotely.

3. Proposed Solution

The proposed system uses RFID technology combined with a microcontroller to automate inventory tracking at construction sites. Every material bundle, tool, or piece of equipment is tagged with a unique passive RFID tag. An RFID reader module (MFRC522) is installed at the store entrance/exit or a designated checkpoint.

When a tagged item passes near the reader, the module reads the tag's unique ID and passes it to the ESP8266 NodeMCU. The ESP8266 checks the received ID against the registered material list, updates the inward/outward inventory count accordingly, displays the material name and transaction status on an LCD screen, and uploads the tag ID, material name, transaction type (IN/OUT), and timestamp to a cloud platform over Wi-Fi.

RFID Tag → RFID Reader (MFRC522) → ESP8266 → LCD Display (local status) + Cloud Upload (remote monitoring)

4. System Architecture

The overall architecture of the system consists of four layers:

Input Layer (Sensing)

RFID Tag attached to each material/tool; RFID Reader Module (MFRC522) reads the tag's unique ID when brought near it.

Processing Layer (Controller)

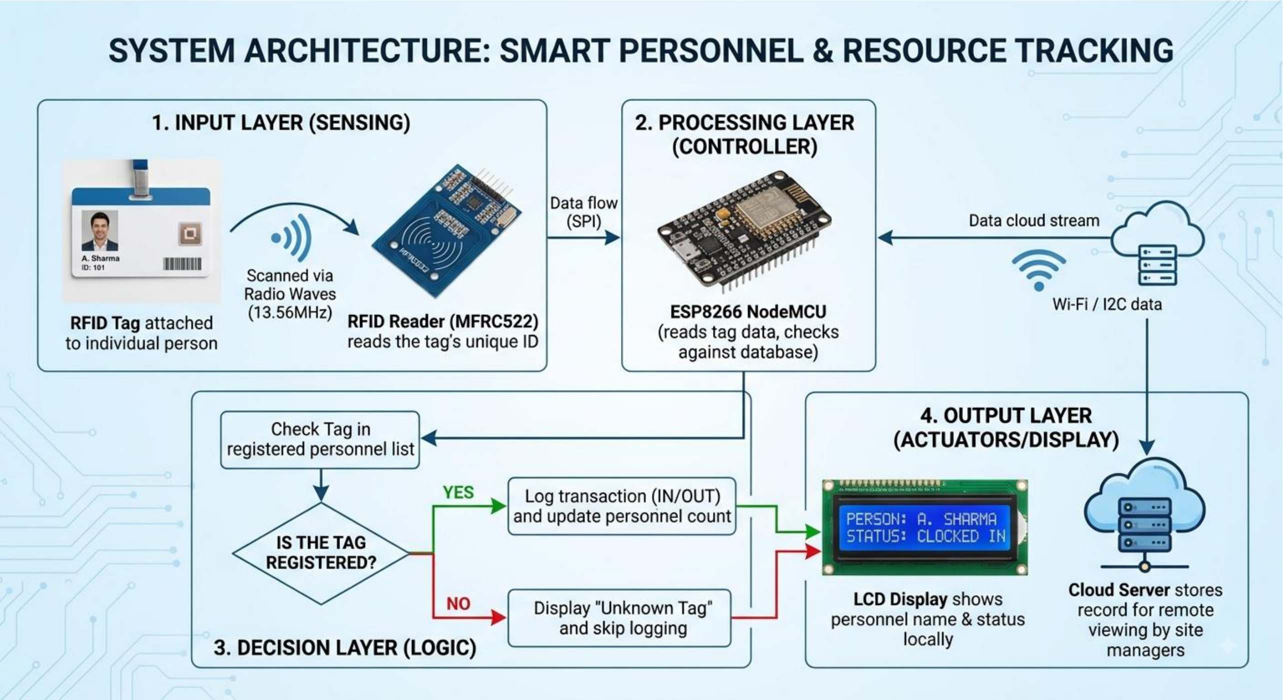
ESP8266 NodeMCU reads the tag data, checks it against the registered material list, and decides whether it is an inward or outward transaction.

Decision Layer (Logic)

Is the tag registered? If YES → log the transaction (IN/OUT) and update the count. If NO → display "Unknown Tag" and skip logging.

Output Layer (Actuators/Display)

LCD Display shows the material name and transaction status locally; the Cloud Server stores the transaction record for remote viewing by project managers.



5. Circuit Table

Component	Pin	ESP8266 NodeMCU
RFID RC522	SDA (SS)	D4
RFID RC522	SCK	D5
RFID RC522	MOSI	D7
RFID RC522	MISO	D6
RFID RC522	RST	D3
RFID RC522	VCC	3V3
RFID RC522	GND	GND
LCD (I2C)	VCC	VIN (5V)
LCD (I2C)	GND	GND
LCD (I2C)	SDA	D2
LCD (I2C)	SCL	D1

Power Supply

The ESP8266 can be powered through USB. The RC522 RFID module operates strictly at 3.3V logic and must never be connected to the 5V rail, as this can damage the module. The I2C LCD module can be powered from the 5V VIN pin of the NodeMCU.

6. Circuit Design

The circuit consists of four major sections:

6.1 ESP8266 NodeMCU

The ESP8266 is the main controller. It communicates with the RFID reader over SPI, drives the LCD over I2C, and connects to Wi-Fi to upload transaction data to the cloud.

6.2 RFID Reader (RC522)

The RC522 module communicates with the ESP8266 over the SPI interface and detects the unique ID of any RFID tag brought within its read range.

RC522 SDA → ESP8266 D4

6.3 RFID Tags

Passive RFID tags or cards are attached to each material bundle, tool, or piece of equipment. Each tag carries a factory-programmed unique identification number used to identify the item.

6.4 LCD Display (16x2 I2C)

The LCD displays the material name, transaction type (IN/OUT), and running inventory count.

LCD SDA → ESP8266 D2 | LCD SCL → ESP8266 D1

6.5 Cloud Connectivity

The ESP8266 connects to the site Wi-Fi network and uploads each transaction record (tag ID, material name, transaction type, timestamp) to a cloud platform such as Firebase Realtime Database or ThingSpeak, enabling remote monitoring by project managers.

7. Algorithm

Algorithm

1. Start the system.
2. Initialize the ESP8266.
3. Initialize the RFID reader module.
4. Initialize the LCD display.
5. Connect the ESP8266 to Wi-Fi and the cloud server.

6. Wait for an RFID tag to be scanned.
7. Read the unique ID of the scanned tag.
8. Check the tag ID against the registered material list.
9. If the tag is registered, determine whether it is an inward or outward transaction.
10. Update the inventory count accordingly.
11. Display the material name and transaction status on the LCD.
12. Upload the tag ID, material name, transaction type, and timestamp to the cloud.
13. If the tag is not registered, display "Unknown Tag" on the LCD.
14. Continue monitoring for the next tag.
15. Repeat the process continuously.

8. Coding

The following Arduino program is designed for the ESP8266 NodeMCU used in the project. It uses the MFRC522 library for RFID, LiquidCrystal_I2C for the display, and the ESP8266WiFi/HTTPClient libraries to push data to the cloud.

```
#include <SPI.h>

#include <MFRC522.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>


// ==================================================

// RC522 RFID - ESP8266 GPIO PINS

// ==================================================


#define SS_PIN  15    // D8

#define RST_PIN 0     // D3


MFRC522 mfrc522(SS_PIN, RST_PIN);
```

```
// ==================================================
// I2C LCD - ESP8266 GPIO PINS
// ==================================================

#define SDA_PIN 4     // D2
#define SCL_PIN 5     // D1

LiquidCrystal_I2C lcd(0x27, 16, 2);


// ==================================================
// SETUP
// ==================================================

void setup() {

  // Serial Monitor
  Serial.begin(9600);
  delay(1000);

  Serial.println();
  Serial.println("==============================");
  Serial.println("       RFID SYSTEM");
  Serial.println("==============================");
```

```
// ==================================================
// START LCD
// ==================================================

Wire.begin(SDA_PIN, SCL_PIN);

lcd.init();
lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("RFID SYSTEM");

lcd.setCursor(0, 1);
lcd.print("Starting...");


// ==================================================
// START RFID
// ==================================================

SPI.begin();

mfrc522.PCD_Init();

delay(1000);
```

```
  // ==================================================
  // READY
  // ==================================================

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("RFID SYSTEM");

  lcd.setCursor(0, 1);
  lcd.print("Scan Your Card");


  Serial.println("LCD Ready");
  Serial.println("RC522 Ready");
  Serial.println("Scan Your Card");
  Serial.println("==============================");
}


// ==================================================
// LOOP
// ==================================================

void loop() {
```

```
// ------------------------------------------------
// Check for new RFID card
// ------------------------------------------------

if (!mfrc522.PICC_IsNewCardPresent()) {
  return;
}


// ------------------------------------------------
// Read RFID card
// ------------------------------------------------

if (!mfrc522.PICC_ReadCardSerial()) {
  return;
}


// ------------------------------------------------
// Create UID string
// ------------------------------------------------

String uid = "";

Serial.print("UID: ");


for (byte i = 0; i < mfrc522.uid.size; i++) {
```

```
  if (mfrc522.uid.uidByte[i] < 0x10) {

    Serial.print("0");
    uid += "0";
  }

  Serial.print(mfrc522.uid.uidByte[i], HEX);

  uid += String(mfrc522.uid.uidByte[i], HEX);
}


uid.toUpperCase();

Serial.println();


// ==================================================
// DOOR
// ==================================================

if (uid == "BA8A8132") {

  showKnownTag("Door");

}
```

```
// ==================================================
// LAB
// ==================================================

else if (uid == "7BABD13E") {

  showKnownTag("Lab");

}



// ==================================================
// SATHEESH
// ==================================================

else if (uid == "D61A9269") {

  showKnownTag("Kavin");

}



// ==================================================
// SABEER
// ==================================================

else if (uid == "6F22CB48") {
```

```
  showKnownTag("Kowz");

}



// ==================================================
// PRASANA
// ==================================================

else if (uid == "1F22EB48") {

  showKnownTag("Hemz");

}



// ==================================================
// UNKNOWN CARD
// ==================================================

else {

  Serial.println("Unknown RFID Tag");
  Serial.println("Access Denied");
  Serial.println("------------------------------");

  lcd.clear();
```

```
  lcd.setCursor(0, 0);
  lcd.print("Unknown Tag");

  lcd.setCursor(0, 1);
  lcd.print("Access Denied");

}


// -----------------------------------------------
// Keep result on LCD
// -----------------------------------------------

delay(3000);


// -----------------------------------------------
// Stop RFID communication
// -----------------------------------------------

mfrc522.PICC_HaltA();
mfrc522.PCD_StopCrypto1();


// -----------------------------------------------
// Ready for next card
// -----------------------------------------------
```

```
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("RFID SYSTEM");

  lcd.setCursor(0, 1);
  lcd.print("Scan Your Card");
}


// ==================================================
// FUNCTION: KNOWN RFID TAG
// ==================================================

void showKnownTag(String name) {

  Serial.print("Known RFID Tag: ");
  Serial.println(name);

  Serial.println("Access Granted");
  Serial.println("------------------------------");


  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("Known Tag");
```

```
  lcd.setCursor(0, 1);
  lcd.print(name);
}
```

Code Explanation

- SS_PIN (D4) and RST_PIN (D3) interface the RC522 reader over SPI.
- The LCD is driven over I2C at address 0x27 (address may vary by module).
- getMaterialName() maps a scanned tag UID to a registered material name — this list should be populated with the actual tag IDs used on site.
- uploadToCloud() sends the tag ID, material name, and IN/OUT status to the cloud endpoint whenever a registered tag is scanned.
- Unregistered tags are shown as "Unknown Tag" on the LCD and are not logged as valid stock movement.
- The tag-to-material mapping and cloud endpoint should be updated to match the actual deployment.

9. System Working

When the system is powered ON, the ESP8266 initializes the RFID reader, the LCD, and connects to the configured Wi-Fi network.

The RFID reader continuously waits for a tag to be scanned as material moves through the checkpoint.

Registered Tag Condition

When a registered tag is scanned:

Tag Scanned → ESP8266 identifies material → Inventory count updated → LCD shows material + IN/OUT status → Record uploaded to cloud

Unregistered Tag Condition

When an unrecognized tag is scanned:

Tag Scanned → ESP8266 finds no match → LCD shows "Unknown Tag" → No inventory update

This process repeats automatically for every item movement, allowing the system to maintain an accurate, continuously updated inventory record both on site and in the cloud.

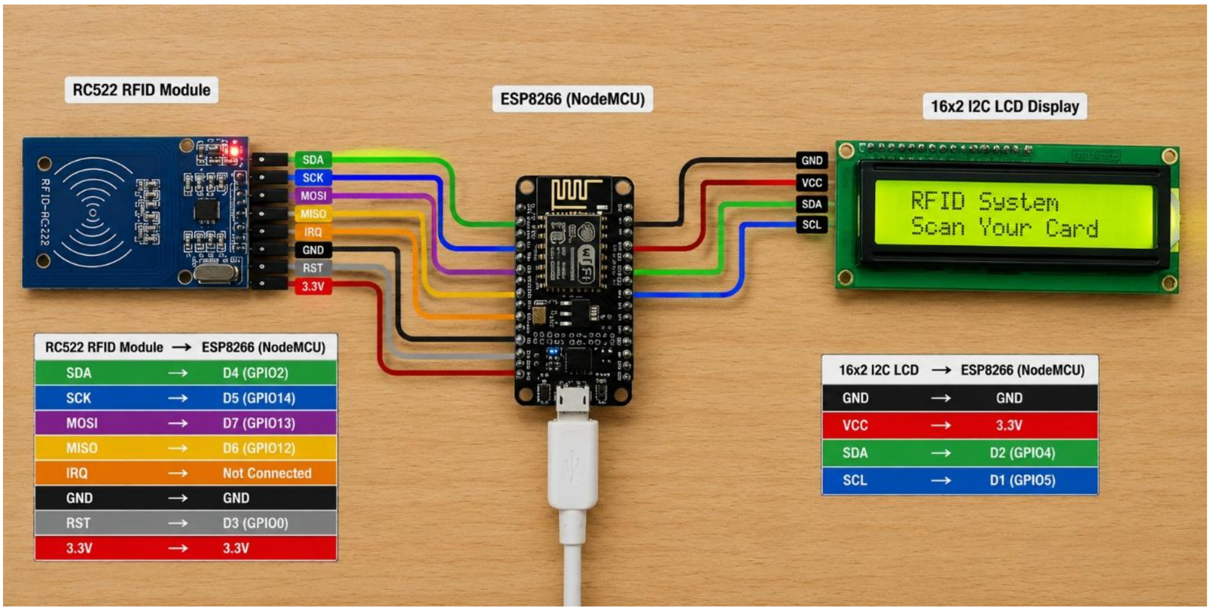
10. Bill of Materials

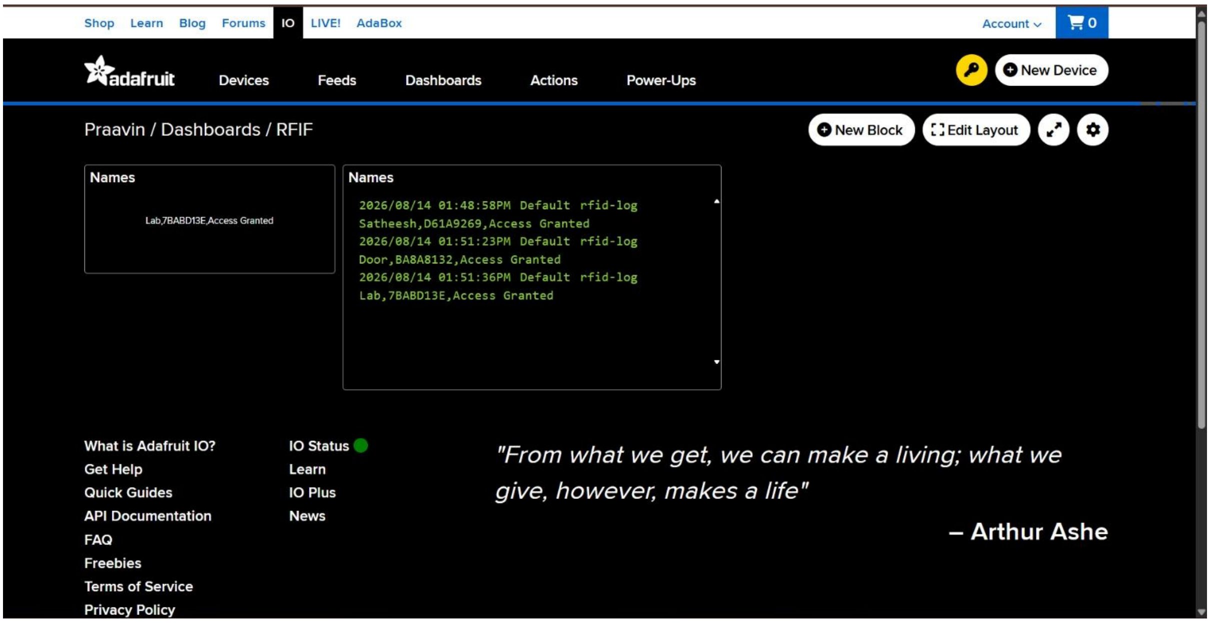
S.No.	Component	Quantity
1	ESP8266 NodeMCU	1
2	RFID Reader Module (MFRC522)	1
3	RFID Tags / Cards	1 set
4	16x2 LCD Display with I2C Module	1
5	Breadboard	1
6	Jumper Wires	1 set
7	USB Power Supply / Battery	1

Software Requirements

- Arduino IDE
- ESP8266 Board Package
- MFRC522 RFID Library
- LiquidCrystal_I2C Library
- Cloud platform account (e.g., Firebase Realtime Database / ThingSpeak)

11. Prototype Implementation





The prototype was designed using an ESP8266 NodeMCU, RFID reader module (RC522), RFID tags, and a 16x2 I2C LCD display. The components were arranged and wired on a breadboard, with the RC522 connected over SPI and the LCD connected over I2C.

The ESP8266 was selected as the main controller because it provides sufficient GPIO for both SPI and I2C interfaces and supports Wi-Fi-based IoT connectivity needed for cloud upload.

The prototype operates automatically: whenever an RFID-tagged material is scanned at the checkpoint, the system identifies the item, updates the LCD, and pushes the transaction to the cloud.

12. Results & Performance Analysis

EXPECTED RESULTS

The system successfully demonstrates automatic inventory tracking based on RFID tag detection.

Condition	Tag Status	LCD Display	Cloud Update
Registered material scanned (inward)	Known	Material name + "IN"	Logged as inward transaction
Registered material scanned (outward)	Known	Material name + "OUT"	Logged as outward transaction
Unregistered / foreign tag scanned	Unknown	"Unknown Tag"	Not logged / flagged

Performance

The system provides the following benefits:

- Automatic, real-time material identification.
- Reduced manual entry and paperwork.
- Reduced material theft, loss, and misplacement.
- Accurate, continuously updated stock records.
- Remote visibility for project managers via the cloud.

Conclusion

The Smart Inventory Management System provides an efficient approach to automated material tracking on construction sites. By using RFID tags on materials and equipment, the system can automatically identify each item as it moves through a checkpoint, update inventory counts, and make the data available both locally and remotely.

The ESP8266 acts as the central controller, the RFID reader identifies tagged materials, the LCD provides on-site status visibility, and the cloud platform enables remote monitoring by project managers and contractors.

The proposed system can help reduce material loss, improve procurement accuracy, and streamline site management. In future development, additional features such as GPS-based multi-site tracking, automated low-stock alerts, mobile applications, weight/quantity sensors for bulk materials, and integration with procurement systems can be incorporated.